

DISTRIBUTED AND CLOUD COMPUTING

LAB 5: RPC - GRPC & JAVA RMI

(Module: Services & API Architectures)



Remote Procedure Call (RPC)

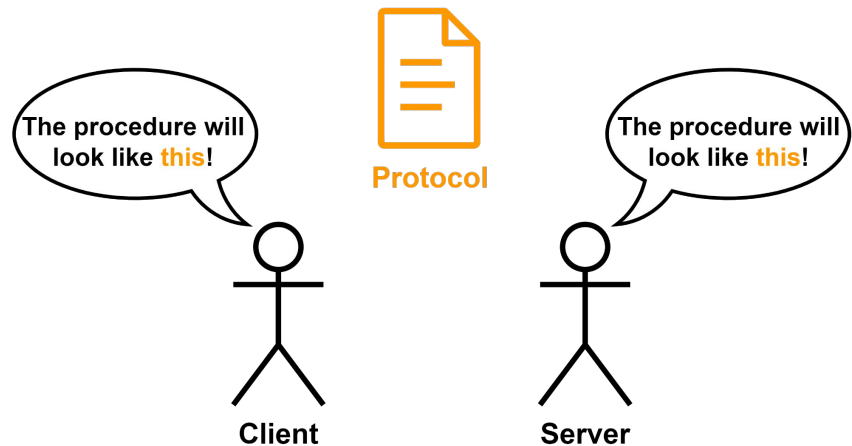
- “In distributed computing, a remote procedure call (RPC) is when a computer program causes a procedure to execute in a different address space (commonly on another computer on a shared computer network), which is written as if it were a normal (local) procedure call, without the programmer explicitly writing the details for the remote interaction.”
 - a. **serve remotely**
 - b. **invoke as if it were local**
 - c. **abstract remote interaction details**
- An intuitive example: Holobox
 - a. the person to talk to is not here
 - b. talk to the person’s hologram
 - c. do not care how the hologram is set up
- What are the **design concerns** for achieving the aforementioned key points?



RPC - Design Concerns

- **Protocol:**

- a. “If the procedure is on a remote server, how do I know its signature to properly execute it?”
- b. Also referred to as Contract, Interface, or **Interface Definition Language (IDL)**.
- c. Procedure signature:
 - Name
 - Input parameters
 - Output format

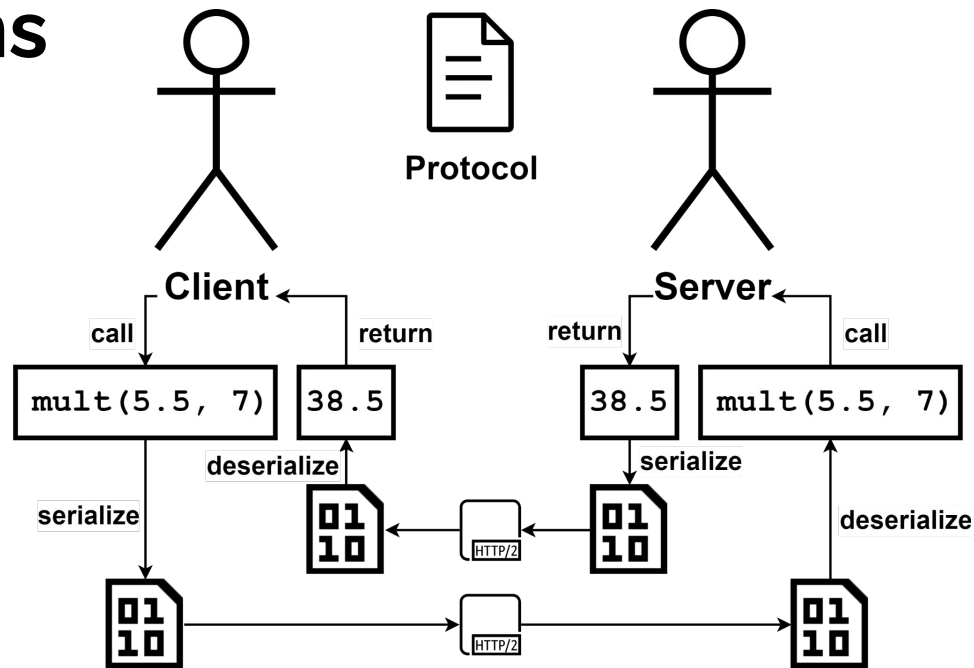


- Specifying the protocol in advance:

- a. allows clients and servers to perform implementations independently;
- b. supports advanced client/server-side code generation for RPC.

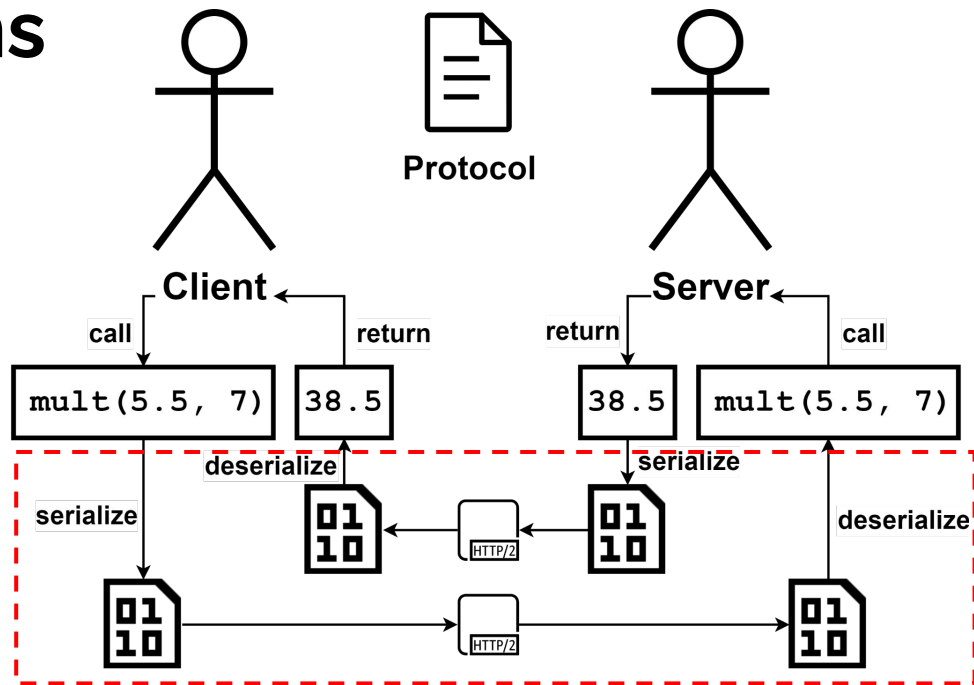
RPC - Design Concerns

- **Protocol**
- **Communication:**
 - a. “How do I connect to the remote server and invoke the remote procedure on it?”
 - b. Also referred to as Transmission.
 - c. **Serialization:** “how do I pack my request parameters into a transmittable format?”
 - d. **Network Protocol:** “do I use TCP or HTTP to manage the internet connection?”
- To reduce the network communication overhead, we might:
 - a. shrink the transmission data size;
 - b. select an appropriate network protocol to manage multiple requests (potentially for multiple RPC procedures) efficiently.



RPC - Design Concerns

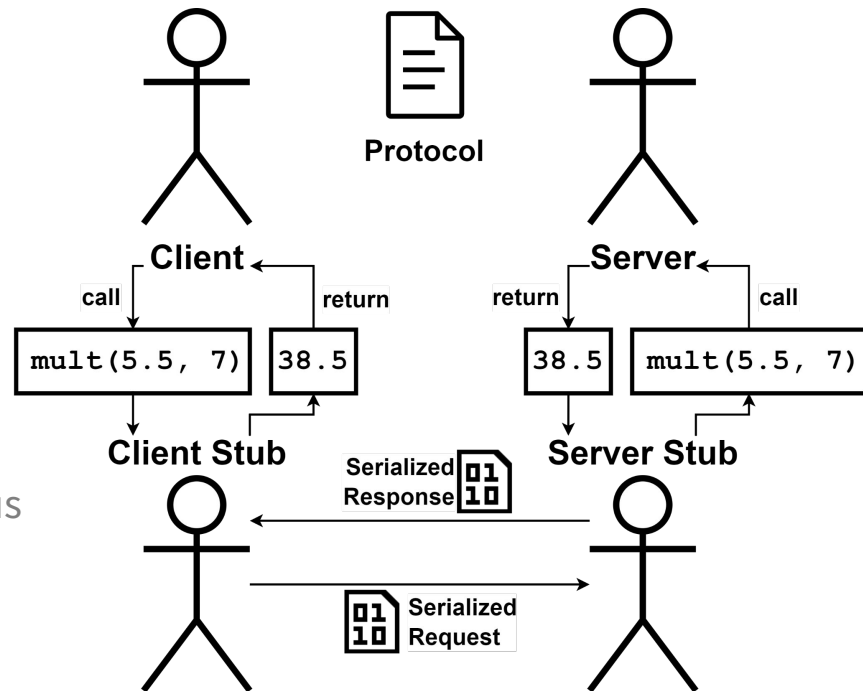
- **Protocol**
- **Communication:**
 - a. “How do I connect to the remote server and invoke the remote procedure on it?”
 - b. Also referred to as Transmission.
 - c. **Serialization:** “how do I pack my request parameters into a transmittable format?”
 - d. **Network Protocol:** “do I use TCP or HTTP to manage the internet connection?”



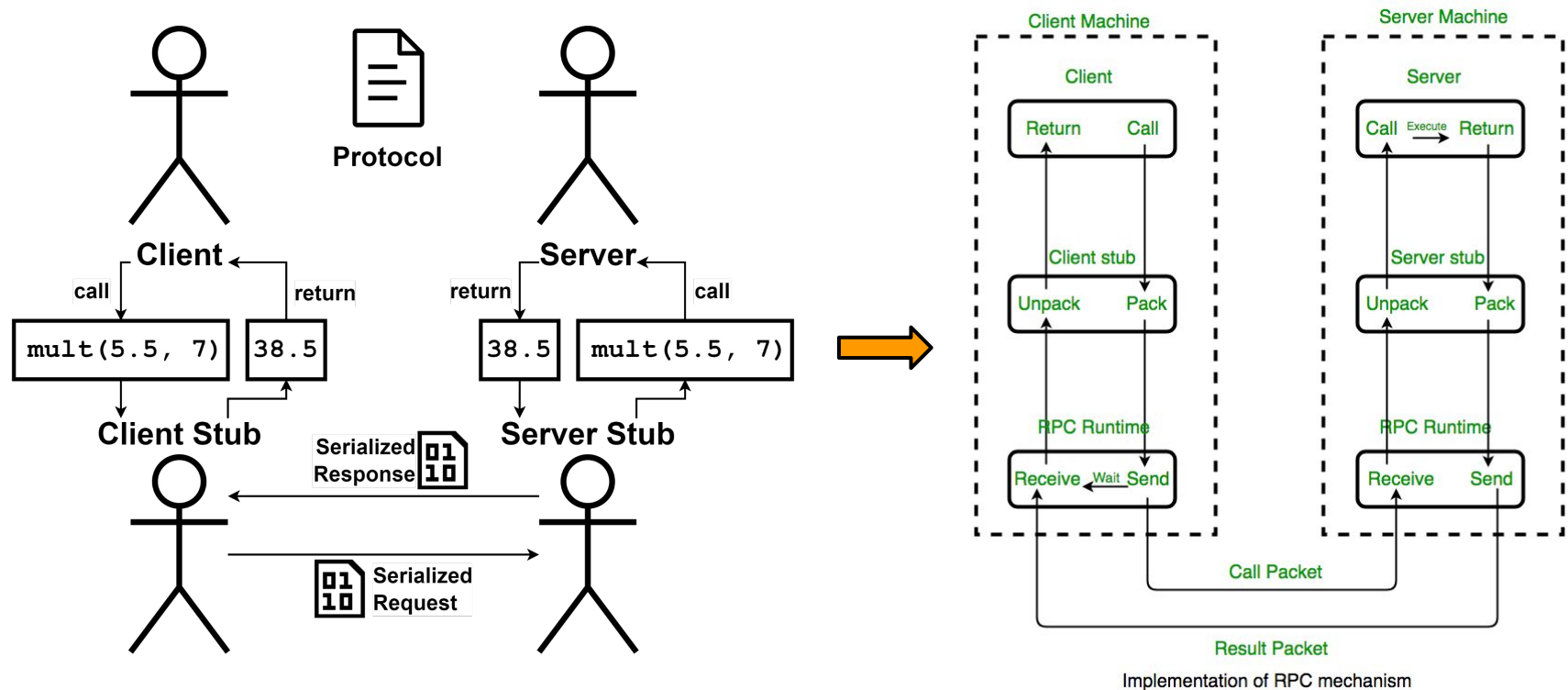
- Communication handling introduces too much complexity to both the client & the server!

RPC - Design Concerns

- **Protocol**
- **Communication**
- **Proxy:**
 - Client Stub:**
 - “Can I just focus on requesting the correct procedure with the correct parameters, while letting someone help me handle the rest?”
 - Also referred to as Client-side Proxy.
 - Server Stub:**
 - “Can the server developers just focus on the procedure implementation, while letting someone help handle the rest?”
 - Also referred to as Server-side Proxy, Skeleton.

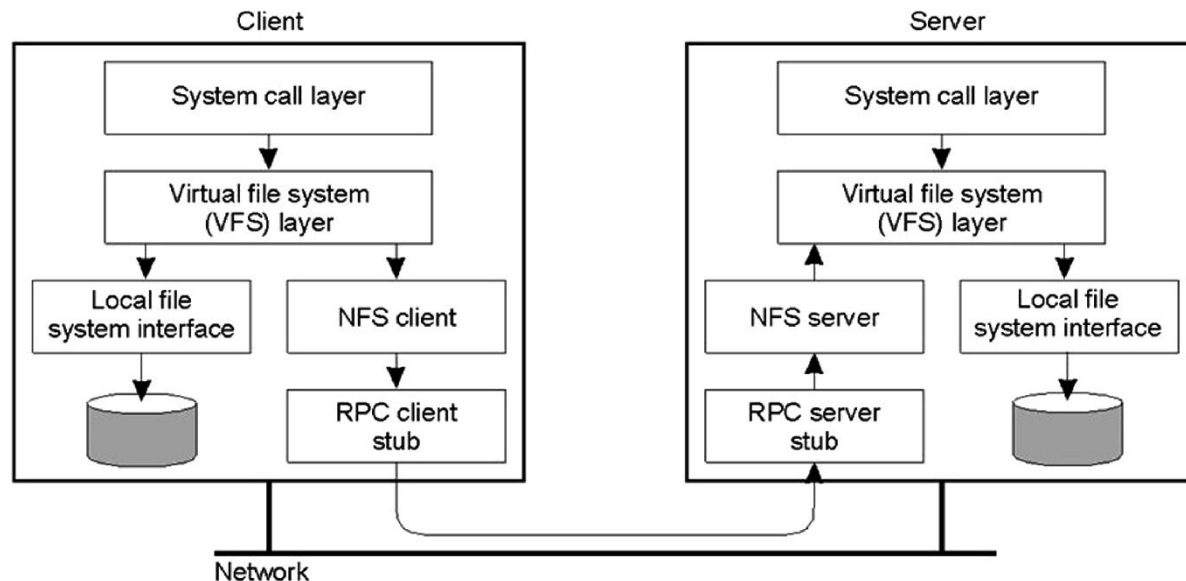


RPC - Common Structure



RPC - Application Scenarios

- **Network File System (NFS):**
 - Makes remote file operations (e.g., file copying) feel local.
 - `cp /mnt/nfs/data.txt ~/Documents/new_data.txt` as usual, RPC will handle the communication details behind the scene.



RPC - Application Scenarios

- **Network File System (NFS):**
 - a. Makes remote file operations (e.g., file copying) feel local.
 - b. `cp /mnt/nfs/data.txt ~/Documents/new_data.txt` as usual, RPC will handle the communication details behind the scene.
- **PyTorch Distributed RPC:**
 - a. Manages multi-machine model training/inference.
 - b. Supports referencing remote objects without copying the real data around.
- **Backend-to-backend Communication in Microservices**
- ...

```
class RNNModel(nn.Module):
    def __init__(self, ps, ntoken, ninp, nhid, nlayers, dropout=0.5):
        super(RNNModel, self).__init__()

        # setup embedding table remotely
        self.emb_table_rref = rpc.remote(ps, EmbeddingTable, args=(ntoken, ninp, dropout))
        # setup LSTM locally
        self.rnn = nn.LSTM(ninp, nhid, nlayers, dropout=dropout)
        # setup decoder remotely
        self.decoder_rref = rpc.remote(ps, Decoder, args=(ntoken, nhid, dropout))

    def forward(self, input, hidden):
        # pass input to the remote embedding table and fetch emb tensor back
        emb = _remote_method(EmbeddingTable.forward, self.emb_table_rref, input)
        output, hidden = self.rnn(emb, hidden)
        # pass output to the remote decoder and get the decoded output back
        decoded = _remote_method(Decoder.forward, self.decoder_rref, output)
        return decoded, hidden
```

gRPC As an RPC Implementation

- “[gRPC](#) is a cross-platform high-performance remote procedure call (RPC) framework, created by Google.”
- “gRPC is a [Cloud Native Computing Foundation \(CNCF\)](#) project.”



Used by



- How does gRPC fulfill the design concerns?
 - Protocol
 - Communication
 - Proxy

hw_client.py X **“abstracts the complexity of network communication”**

```
0_rpc > 0_grpc > 0_hello_world > hw_client.py > ...
1 import grpc
2 from assistant_pb2 import GreetRequest, MultRequest
3 from assistant_pb2_grpc import AssistantServiceStub
4
5 def run():
6     with grpc.insecure_channel('localhost:8082') as channel:
7         stub = AssistantServiceStub(channel)
8         # Greeting
9         print('> Greet: Hello Assistant?')
10        res = stub.GreetWithInfo(GreetRequest(user_name='Peter', institution='SUSTech'))
11        print(f'> Client received:\n(res)')
12        # Multiplication
13        print('> Mult: Requesting a multiplication task')
14        res = stub.Multiply(MultRequest(xin=3.5, yin=5))
15        print(f'> Client received:\n(res)')
16
17 if __name__ == '__main__':
18     run()
19
```

An orange arrow points from the text “abstracts the complexity of network communication” to the line `with grpc.insecure_channel('localhost:8082') as channel:` in the code block.

TASK: gRPC - Hello World

Implement a Hello World gRPC Python example.

> Reference codebase: `rpc_grpc_hello_world`

1. Set up Python ([uv](#) is recommended).
2. Install Python dependencies into a uv environment via:
 - `uv sync`
3. Check the protocol file `assistant.proto`. Use `protoc` to generate some code:
 - `uv run python -m grpc_tools.protoc -I./ --python_out=. --pyi_out=. --grpc_python_out=. assistant.proto`
 - These generated code will be utilized to implement the gRPC client and the gRPC server.

```

assistant.proto
0_rpc > 0_rpc > 0_hello_world > assistant.proto > ...
1 syntax = "proto3";
2
3 // Style Guide: https://protobuf.dev/programming-guides/style/.
4 // Files should be named "lower_snake_case.proto".
5
6 /*
7 Services are what the servers provide for the clients. Specifically for gRPC.
8 Use PascalCase (with an initial capital) for both the service name and any RPC method names.
9 */
10 service AssistantService {
11   // Constructs a greeting message based on the given information of the user.
12   rpc GreetWithInfo(GreetRequest) returns (GreetResponse);
13   // Multiply two given numbers and give back the output (with the inputs).
14   rpc Multiply(MultRequest) returns (MultResponse);
15 }
16
17 /*
18 Messages are exchanged between clients and servers.
19 Use PascalCase (with an initial capital) for message names: SongServerRequest.
20 Prefer to capitalize abbreviations as single words: GetDnsRequest rather than GetDNSRequest.
21 Use lower_snake_case for field names, including oneof field and extension names: song_name.
22 */
23 // The greeting request message with the user's name and institution.
24 message GreetRequest {
25   string user_name = 1; // user's name at the 1st position
26   string institution = 2; // user's institution at the 2nd position
27 }
28
29 // The greeting response message with the constructed message.
30 message GreetResponse {
31   string message = 1;
32 }
33
34 // The multiplication request message including two input double numbers.
35 message MultRequest {
36   double xin = 1;
37   double yin = 2;
38 }
39
40 // The multiplication response message including the inputs and the output number.
41 message MultResponse {
42   double xin = 1;
43   double yin = 2;
44   double result = 3;
45 }
46

```

```

/tmp/rpc_rest/0_rpc/0_grpc/0_hello_world/hello_world # ls -lh
Makefile
README.md
assistant.proto
assistant_pb2.py
assistant_pb2.pyi
assistant_pb2_grpc.py
hw_client.py
hw_server.py
pyproject.toml
requirements.txt
uv.lock

```

protoc

TASK: gRPC - Hello World

Implement a Hello World gRPC Python example.

> Reference codebase: [rpc_grpc_hello_world](#)

1. Set up Python ([uv](#) is recommended).
2. Install Python dependencies into a uv environment via:
 - `uv sync`
3. Use `protoc` to generate some code.
4. Implement the gRPC server in `hw_server.py`. Run the gRPC server via:
 - `uv run python hw_server.py`
5. Implement the gRPC client in `hw_client.py`. In another terminal, run the gRPC client via:
 - `uv run python hw_client.py`

```
root@RAINBOW: /rainbow/ai/ai-llm/ai-llm-101/grpc_hello_world# uv run python hw_server.py
INFO:root:Server started, listening on 8082
```

```
root@RAINBOW: /rainbow/ai/ai-llm/ai-llm-101/grpc_hello_world# uv run python hw_client.py
> Greet: Hello Assistant?
> Client received:
message: "Hello Peter from SUSTech!"

> Mult: Requesting a multiplication task
> Client received:
xin: 3.5
yin: 5
result: 17.5
```

```
hw_server.py
0_rpc > 0_grpc > 0_hello_world > hw_server.py > ...
1 from concurrent import futures
2 import logging
3
4 import grpc
5 from assistant_pb2_grpc import AssistantServiceServicer, add_AssistantServiceServicer_to_server
6 from assistant_pb2 import GreetRequest, GreetResponse, MultRequest, MultResponse
7
8 class Assistant(AssistantServiceServicer):
9     def GreetWithInfo(self, request: GreetRequest, context):
10         msg = f'Hello {request.user_name} from {request.institution}!'
11         return GreetResponse(message=msg)
12
13     def Multiply(self, request: MultRequest, context):
14         res = request.xin * request.yin
15         return MultResponse(xin=request.xin, yin=request.yin, result=res)
16
17 def serve():
18     port = '8082'
19     # the server can handle 10 client requests concurrently
20     server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
21     add_AssistantServiceServicer_to_server(Assistant(), server)
22     # [:::] specifies the listen on all ipv4/ipv6 addresses
23     server.add_insecure_port('[:::]' + port)
24     server.start()
25     logging.info(f'Server started, listening on {port}')
26     server.wait_for_termination()
27
28 if __name__ == '__main__':
29     logging.basicConfig(level=logging.INFO)
30     serve()
31
```

“abstracts the complexity of network communication”

```
hw_client.py
0_rpc > 0_grpc > 0_hello_world > hw_client.py > ...
1 import grpc
2 from assistant_pb2 import GreetRequest, MultRequest
3 from assistant_pb2_grpc import AssistantServiceStub
4
5 def run():
6     with grpc.insecure_channel('localhost:8082') as channel:
7         stub = AssistantServiceStub(channel)
8         # Greeting
9         print('> Greet: Hello Assistant?')
10        res = stub.GreetWithInfo(GreetRequest(user_name='Peter', institution='SUSTech'))
11        print(f'> Client received:\n{res}')
12        # Multiplication
13        print('> Mult: Requesting a multiplication task')
14        res = stub.Multiply(MultRequest(xin=3.5, yin=5))
15        print(f'> Client received:\n{res}')
16
17 if __name__ == '__main__':
18     run()
19
```

TASK: gRPC - Quick Start with Go

Hello World again, but this time with Go.

We will also use a bit of [Go](#) in our future lab sessions to demonstrate the cross-language feature of gRPC. Set up Go and try the [official quick start guide](#).

```
root@RAINBOW:~# go version
go version go1.25.1 linux/amd64
```

1. Install Go according to the [official documentation](#). Verify with `go version`.
2. (optional) Set a proxy for Go to smoothly download the dependencies:
 - `export GOPROXY=https://goproxy.cn,direct`
3. Install the Protobuf compiler, `protoc`, version 3 according to the [official documentation](#).
4. Install Go plugins for Protobuf:
 - `go install google.golang.org/protobuf/cmd/protoc-gen-go@latest`
 - `go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest`
 - These plugins will be installed to a `bin/` folder from a preset `GOPATH`. Add this folder to the system path and `protoc` will be able to find these plugins:

```
root@RAINBOW:~# go env GOPATH
/root/go
```

 - `export PATH="$PATH:$(go env GOPATH)/bin"`
5. Clone the official gRPC Go GitHub repository and enter the Hello World example folder:
 - `git clone -b v1.75.0 --depth 1 https://github.com/grpc/grpc-go`
 - `cd grpc-go/examples/helloworld`

TASK: gRPC - Quick Start with Go

Hello World again, but this time with Go.

We will also use a bit of [Go](#) in our future lab sessions to demonstrate the cross-language feature of gRPC. Set up Go and try the [official quick start guide](#).

6. Check the proto file:
`helloworld/helloworld.proto`. Some Go code has already been generated by protoc within the same directory. To keep it simple, we will not generate them again for this lab session.
7. Run the server code:
 - `go run greeter_server/main.go`
8. In another terminal, run the client code:
 - `go run greeter_client/main.go`

```
(base) root@RAINBOW: /rainbow/extra/lab/docs/grpc-go/examples/helloworld# go run greeter_server/main.go
go: downloading google.golang.org/genproto v0.0.0-20240814211410-ddb44dafa142
2024/10/09 06:22:37 server listening at [::]:50051
```

```
(base) root@RAINBOW: /rainbow/extra/lab/docs/grpc-go/examples/helloworld# go run greeter_client/main.go
2024/10/09 06:30:34 Greeting: Hello world
```

```
helloworld.proto x
helloworld > helloworld.proto > ...

15 syntax = "proto3";
16
17 option go_package = "google.golang.org/grpc/examples/helloworld/helloworld";
18 option java_multiple_files = true;
19 option java_package = "io.grpc.examples.helloworld";
20 option java_outer_classname = "HelloWorldProto";
21
22 package helloworld;
23
24 // The greeting service definition.
25 service Greeter {
26   // Sends a greeting
27   rpc SayHello (HelloRequest) returns (HelloReply) {}
28 }
29
30 // The request message containing the user's name.
31 message HelloRequest {
32   string name = 1;
33 }
34
35 // The response message containing the greetings
36 message HelloReply {
37   string message = 1;
38 }
```


TASK: gRPC - Quick Start with Go

Hello World again, but this time with Go.

We will also use a bit of [Go](#) in our future lab sessions to demonstrate the cross-language feature of gRPC. Set up Go and try the [official quick start guide](#).

6. Check the proto file:
[helloworld/helloworld.proto](#). Some Go code has already been generated by protoc within the same directory. To keep it simple, we will not generate them again for this lab session.
7. Run the server code:
 - `go run greeter_server/main.go`
8. In another terminal, run the client code:
 - `go run greeter_client/main.go`

```
(base) root@RAINBOW: ~/go/src/github.com/grpc-ecosystem/grpc-hello-world/helloworld # go run greeter_client/main.go
2024/10/09 06:27:57 could not greet: rpc error: code = DeadlineExceeded desc = context deadline exceed
exit status 1
```

Q: getting DeadlineExceeded?

```
main.go  X
greeter_client > main.go > ...
19 // Package main implements a client for Greeter service.
20 package main
21
22 import (
23     "context"
24     "flag"
25     "log"
26     "time"
27
28     "google.golang.org/grpc"
29     "google.golang.org/grpc/credentials/insecure"
30     pb "google.golang.org/grpc/examples/helloworld/helloworld"
31 )
32
33 const (
34     defaultName = "world"
35 )
36
37 var (
38     addr = flag.String("addr", "127.0.0.1:50051", "the address to connect to")
39     name = flag.String("name", defaultName, "Name to greet")
40 )
41
42 func main() {
43     flag.Parse()
44     // Set up a connection to the server.
45     conn, err := grpc.NewClient(*addr, grpc.WithTransportCredentials(insecure.NewCredentials()))
46     if err != nil {
47         log.Fatalf("did not connect: %v", err)
48     }
49     defer conn.Close()
50     c := pb.NewGreeterClient(conn)
51
52     // Contact the server and print out its response.
53     ctx, cancel := context.WithTimeout(context.Background(), 15*time.Second)
54     defer cancel()
55     r, err := c.SayHello(ctx, &pb.HelloRequest{Name: *name})
56     if err != nil {
57         log.Fatalf("could not greet: %v", err)
58     }
59     log.Printf("Greeting: %s", r.GetMessage())
60 }
61
```

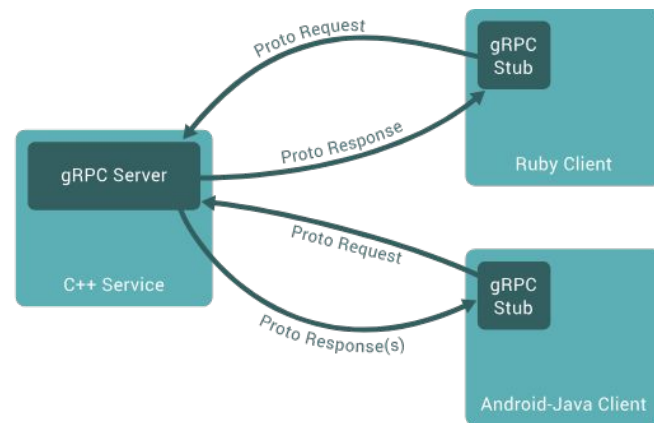
gRPC - Protocol Definition

- gRPC uses **proto files** to define:
 - messages
 - field property (e.g., **repeated**)
 - field type (e.g., **string**)
 - field name
 - field tag/number: ordering of fields
 - remote procedures
 - name
 - request message
 - response message
 - services
 - organized set of remote procedures
- Proto files are a part of Protocol Buffers (**Protobuf**). We will introduce a bit more about Protobuf in the next lab session.

```
assistant.proto X
0_rpc > 0_grpc > 0_hello_world > assistant.proto > ...
1 syntax = "proto3";
2
3 // Style Guide: https://protobuf.dev/programming-guides/style/.
4 // Files should be named "lower_snake_case.proto".
5
6 /*
7  Services are what the servers provide for the clients. Specifically for gRPC.
8  Use PascalCase (with an initial capital) for both the service name and any RPC method names.
9  */
10 service AssistantService {
11     // Constructs a greeting message based on the given information of the user.
12     rpc GreetWithInfo(GreetRequest) returns (GreetResponse);
13     // Multiply two given numbers and give back the output (with the inputs).
14     rpc Multiply(MultRequest) returns (MultResponse);
15 }
16
17 /*
18  Messages are exchanged between clients and servers.
19  Use PascalCase (with an initial capital) for message names: SongServerRequest.
20  Prefer to capitalize abbreviations as single words: GetDnsRequest rather than GetDNSRequest.
21  Use lower_snake_case for field names, including oneof field and extension names: song_name.
22  */
23 // The greeting request message with the user's name and institution.
24 message GreetRequest {
25     string user_name = 1; // user's name at the 1st position
26     string institution = 2; // user's institution at the 2nd position
27 }
28
29 // The greeting response message with the constructed message.
30 message GreetResponse {
31     string message = 1;
32 }
33
34 // The multiplication request message including two input double numbers.
35 message MultRequest {
36     double xin = 1;
37     double yin = 2;
38 }
39
40 // The multiplication response message including the inputs and the output number.
41 message MultResponse {
42     double xin = 1;
43     double yin = 2;
44     double result = 3;
45 }
46
```


gRPC - Protocol Definition

- gRPC uses **proto files** to define:
 - messages
 - remote procedures
 - services
- Proto files serve as the **Interface Definition Language (IDL)** for RPC.
 - “a language that lets a program written in one language communicate with another program written in an unknown language.”
 - Example: a gRPC server is implemented in C++. It can interact with a Ruby gRPC client, or a Java gRPC client from Android platform.



gRPC - Protocol Definition

- gRPC uses **proto files** to define:
 - messages
 - remote procedures
 - services
- Proto files serve as the Interface Definition Language (IDL) for RPC.
 - “a language that lets a program written in one language communicate with another program written in an unknown language.”
- Proto files can be compiled by **protoc** to generate code from different programming languages:

- message classes
- gRPC client stub
- gRPC server template

```

cc/dncc-lab/rpc_test/0_rpc/0_hello_world# ls -lh
Makefile
README.md
__pycache__
assistant.proto
assistant_pb2.py
assistant_pb2.pyi
assistant_pb2_grpc.py
hw_client.py
hw_server.py
requirements.txt
  
```

← protoc

```

assistant_pb2.pyi X
0_rpc > 0_rpc > 0_hello_world > assistant_pb2.pyi > ...

7   class GreetRequest(_message.Message):
8       __slots__ = ("user_name", "institution")
9       USER_NAME_FIELD_NUMBER: _ClassVar[int]
10      INSTITUTION_FIELD_NUMBER: _ClassVar[int]
11      user_name: str
12      institution: str
13      def __init__(self, user_name: _Optional[str] = ...
14
  
```

```

assistant_pb2_grpc.py X
0_rpc > 0_rpc > 0_hello_world > assistant_pb2_grpc.py > AssistantServiceStub > _init_

28  class AssistantServiceStub(object):
29      """Stlye Guide: https://protobuf.dev/programming-guides/style/...
36
37      def __init__(self, channel):
38          """Constructor. ...
43          self.GreetWithInfo = channel.unary_unary(
44              '/AssistantService/GreetWithInfo',
45              request_serializer=assistant_pb2.GreetRequest.SerializeToString,
46              response_deserializer=assistant_pb2.GreetResponse.FromString,
47              _registered_method=True)
  
```

```

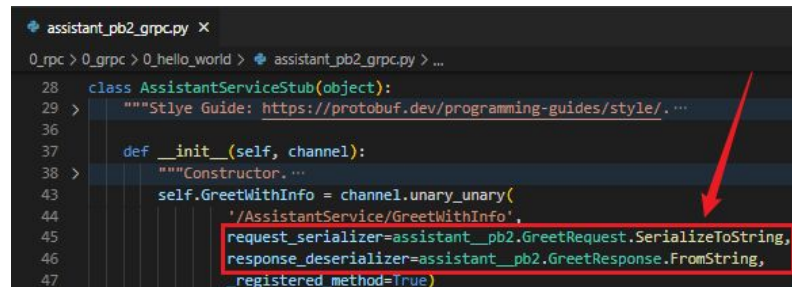
assistant_pb2_grpc.py X
0_rpc > 0_rpc > 0_hello_world > assistant_pb2_grpc.py > AssistantServiceServicer

55  class AssistantServiceServicer(object):
56      """Stlye Guide: https://protobuf.dev/programming-guides/s
63
64      def GreetWithInfo(self, request, context):
65          """Constructs a greeting message based on the given i
66          """
67          context.set_code(grpc.StatusCode.UNIMPLEMENTED)
68          context.set_details('Method not implemented!')
69          raise NotImplementedError('Method not implemented!')
  
```

gRPC - Communication Handling

- **Serialization:**

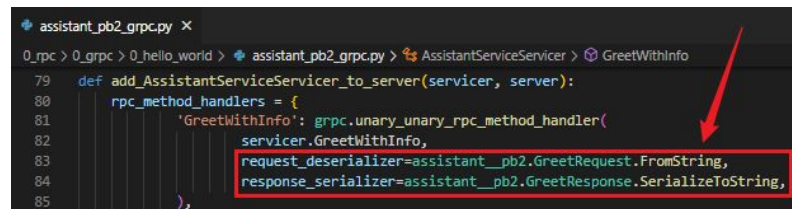
- Messages defined in the proto files are transmitted between the gRPC client and server.
- gRPC uses **Protocol Buffers (Protobuf)** to serialize proto messages into binaries.
- “Protocol Buffers are language-neutral, platform-neutral extensible mechanisms for serializing structured data.”
 - provides more compact data;
 - enables faster transmission.



```

assistant_pb2_grpc.py X
0_rpc > 0_grpc > 0_hello_world > assistant_pb2_grpc.py > ...
28 class AssistantServiceStub(object):
29     """Stlye Guide: https://protobuf.dev/programming-guides/style/. ...
36
37     def __init__(self, channel):
38         """Constructor. ...
43         self.GreetWithInfo = channel.unary_unary(
44             '/AssistantService/GreetWithInfo',
45             request_serializer=assistant_pb2.GreetRequest.SerializeToString,
46             response_deserializer=assistant_pb2.GreetResponse.FromString,
47             _registered_method=True)

```



```

assistant_pb2_grpc.py X
0_rpc > 0_grpc > 0_hello_world > assistant_pb2_grpc.py > AssistantServiceServer > GreetWithInfo
79 def add_AssistantServiceServer_to_server(servicer, server):
80     rpc_method_handlers = {
81         'GreetWithInfo': grpc.unary_unary_rpc_method_handler(
82             servicer.GreetWithInfo,
83             request_serializer=assistant_pb2.GreetRequest.FromString,
84             response_serializer=assistant_pb2.GreetResponse.SerializeToString,
85         ),

```

- We will further examine how Protobuf serializes messages in the next lab session.

serialize_and_deserialize

> Initial Message:

```

user_name: "Peter"
institution: "SUSTech"

```

> After Serialization: b'\n\x05Peter\x12\x07SUSTech'

```

>> Binary form: 00001010 00000101 01010000 01100101 01110100 01100101 01110010 00
010010 00000111 01010011 01010101 01010011 01010100 01100101 01100011 01101000
>> Hex Representation: 0a 05 50 65 74 65 72 12 07 53 55 53 54 65 63 68

```

gRPC - Communication Handling

- **Serialization:**

- Messages defined in the proto files are transmitted between the gRPC client and server.
- gRPC uses **Protocol Buffers (Protobuf)** to serialize proto messages into binaries.

- **Network Protocol:**

- gRPC transmits serialized request/response messages over **HTTP/2 framing**.
- A **channel** represents an HTTP/2 connection (a TCP connection behind the scene).
- A channel manages multiple logical HTTP/2 **streams**, each dedicated to one procedure.
- Messages are sent on the corresponding HTTP/2 streams as HTTP/2 **frames**.



```

hw_client.py x
0_rpc > 0_grpc > 0_hello_world > hw_client.py > run
1 import grpc
2 from assistant_pb2 import GreetRequest, MultRequest
3 from assistant_pb2_grpc import AssistantServiceStub
4
5 def run():
6     with grpc.insecure_channel('localhost:8082') as channel:
7         stub = AssistantServiceStub(channel)
8         # Greeting
9         print('> Greet: Hello Assistant?')
10        res = stub.GreetWithInfo(GreetRequest(user_name='Peter', institution='SUSTech'))
11        print(f'Client received:\n{res}')
  
```

gRPC - Proxy

- Client and server stubs are pre-generated via **protoc** in a static fashion.
 - message classes
 - gRPC client stub
 - The client stub is retrieved from the gRPC channel.
 - The client calls the remote procedure via the client stub.
 - gRPC server template
 - The server extends the generated template and provides the procedure implementation.
 - The server registers the implemented procedure.

```
hw_client.py X
0_rpc > 0_grpc > 0_hello_world > hw_client.py ...
1 import grpc
2 from assistant_pb2 import GreetRequest, MultRequest
3 from assistant_pb2_grpc import AssistantServiceStub
4
5 def run():
6     with grpc.insecure_channel('localhost:8082') as channel:
7         stub = AssistantServiceStub(channel)
8         # Greeting
9         print('> Greet: Hello Assistant?')
10        res = stub.GreetWithInfo(GreetRequest(user_name='Peter', institution='SUSTech'))
11        print(f'> Client received:\n{res}')
```

```
hw_server.py X
0_rpc > 0_grpc > 0_hello_world > hw_server.py ...
1 from concurrent import futures
2 import logging
3
4 import grpc
5 from assistant_pb2_grpc import AssistantServiceServicer, add_AssistantServiceServicer
6 from assistant_pb2 import GreetRequest, GreetResponse, MultRequest,
7
8 class Assistant(AssistantServiceServicer):
9     def GreetWithInfo(self, request: GreetRequest, context):
10         msg = f'Hello {request.user_name} from {request.institution}!'
11         return GreetResponse(message=msg)
```

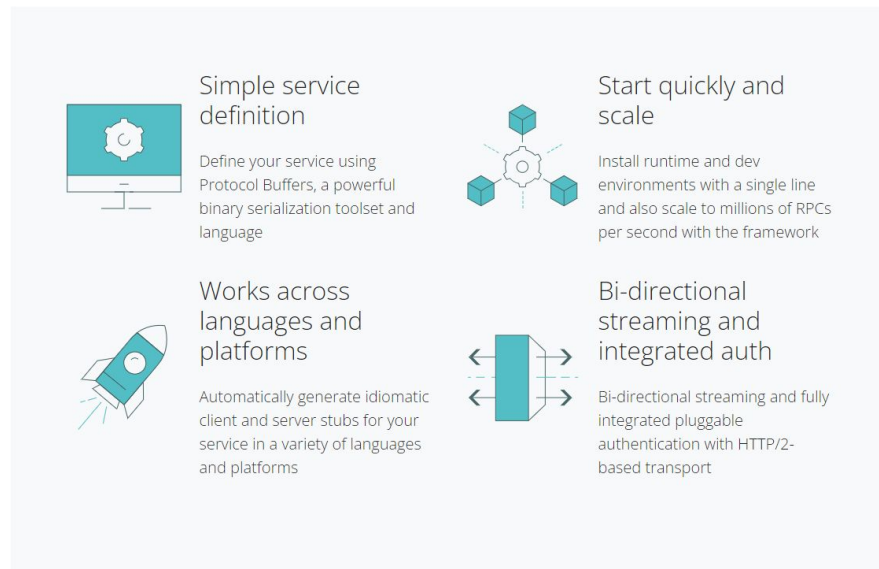
```
hw_server.py X
0_rpc > 0_grpc > 0_hello_world > hw_server.py > serve
17 def serve():
18     port = '8082'
19     # the server can handle 10 client requests concurrently
20     server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
21     add_AssistantServiceServicer_to_server(Assistant(), server)
22     # [::] specifies the listen on all ipv4/ipv6 addresses
23     server.add_insecure_port('[::]:' + port)
24     server.start()
25     logging.info(f'Server started, listening on {port}')
26     server.wait_for_termination()
```

gRPC - Benefits

Why is gRPC so popular:

1. Static & automatic generation of client stubs & server templates via protoc.
2. Efficient communication via Protobuf.
3. Cross-Language + Cross-Platform support.
4. Bidirectional Streaming.
5. Multiplexing via HTTP/2.
6. ...

- More details in the next lab session.



TASK: gRPC - Build Your Own gRPC Service

Refactor a local procedure into a remote procedure.

> Reference codebase: [rpc_grpc_byogrpc](#)

Requirements:

1. The remote procedure should be hosted by the **AssistantService** in the gRPC server.
2. A gRPC client should be written to test the remote procedure.

Reference Steps:

1. Write the proto file.
2. Generate code stubs via protoc.
3. Implement and run the gRPC server.
4. Implement and run the gRPC client.

```
local_procedure.py X
0_rpc > 0_grpc > 0x_build_your_own_grpc_service > local_procedure.py > ...
1  ...
2  TASK: refactor this local procedure 'report_stats' into a gRPC procedure. This procedure belongs to a
3  gRPC service called 'AssistantService'. Then, use a gRPC client to test the remote procedure.
4
5  Hints:
6  1. Python 'float' is FP64.
7  2. For array/list types, check: https://protobuf.dev/programming-guides/proto3/#field-labels
8  ...
9
10 def report_stats(user_name: str, institution: str, values: list[float]) -> tuple[float, float, float, str]:
11     ...
12     report_add calculates the min, max, avg of the provided numeric values, and gives a response
13     in natural language.
14     ...
15     val_min = min(values)
16     val_max = max(values)
17     val_avg = sum(values) / len(values)
18     resp_msg = f'Hi {user_name} from {institution}! For {values}: min={val_min}; max={val_max}, avg={val_avg}'
19     return val_min, val_max, val_avg, resp_msg
20
21 if __name__ == '__main__':
22     # (-3, 5, 0.2, 'Hi Peter S from SUSTech! For [1, 1.5, 2, 2.5, 5, -1, -1.5, -2, -2.5, -3]: min=-3; max=5, avg=0.2')
23     print(report_stats(
24         user_name='Peter S', institution='SUSTech',
25         values=[1, 1.5, 2, 2.5, 5, -1, -1.5, -2, -2.5, -3],
26     ))
```

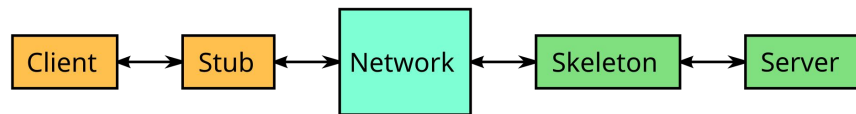
```
> Request:
user_name: "Peter S"
institution: "SUSTech"
values: 1
values: 1.5
values: 2
values: 2.5
values: 5
values: -1
values: -1.5
values: -2
values: -2.5
values: -3

> Response:
min_value: -3
max_value: 5
avg_value: 0.2
message: "Hi Peter S from SUSTech! For [1.0, 1.5, 2.0, 2.5, 5.0, -1.0, -1.5, -2.0, -2.5, -3.0]: min=-3.0; max=5.0, avg=0.2"
```

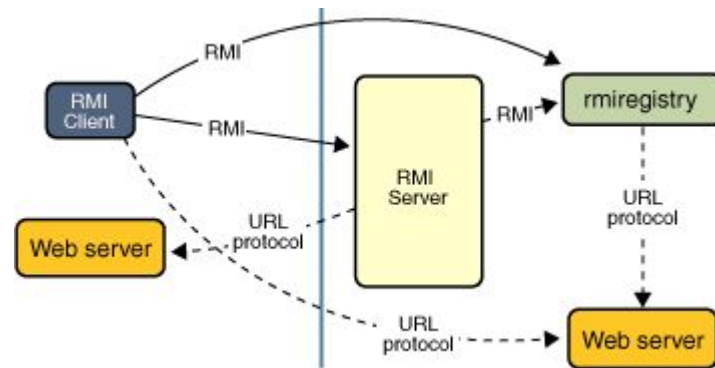
Try starting from scratch to be familiar with the process before Assignment 2.

Java RMI As Another RPC Implementation

- “Java Remote Method Invocation ([Java RMI](#)) enables the programmer to create distributed Java technology-based to Java technology-based applications, in which the methods of remote Java objects can be invoked from other Java virtual machines, possibly on different hosts.”
- Java RMI is a **Java-specific Object-Oriented** implementation of the RPC protocol.



- We introduced only Java RMI before 2024, but since gRPC has now become one of the most popular RPC solutions, we will put more focus on gRPC from now on.
- Nevertheless, it is still helpful to check Java RMI and think about why gRPC receives more flowers.



TASK: Java RMI - Hello World

Implement a Hello World Java RMI example.

> Reference codebase: `rpc_javarmi_hello_world`

1. Set up Java Development Kit (JDK) via:
 - `apt install openjdk-17-jdk`
 - Java RMI is a built-in library, and RMI Registry is natively implemented in the `rmiregistry` program.
2. Check the remote interface definition file

`AssistantService.java`.

- A Java interface is defined with two remote functions.
- The outputs are simplified to one single value, since Java cannot handle multiple output values naturally. To refine, define separate message classes.

```
J AssistantService.java X
rpc_rest > 0_rpc > 1_java_rmi > protocol > J AssistantService.java > ...
1 package protocol;
2
3 import java.rmi.Remote;
4 import java.rmi.RemoteException;
5
6 public interface AssistantService extends Remote {
7     String greetWithInfo(String userName, String institution) throws RemoteException;
8
9     double multiply(double xin, double yin) throws RemoteException;
10 }
11
12 /**
13  * To completely mimic the behavior of the proto file, where remote procedures
14  * may have multiple inputs and outputs, define the message classes manually in
15  * separate files from this package. Requests may be skipped since Java handles
16  * multiple inputs naturally.
17  */
```

TASK: Java RMI - Hello World

Implement a Hello World Java RMI example.

> Reference codebase: [rpc_javarmi_hello_world](#)

1. Set up Java Development Kit (JDK) via:
 - apt install openjdk-17-jdk
 - Java RMI is a built-in library, and RMI Registry is natively implemented in the rmiregistry program.
2. Check the remote interface definition file `AssistantService.java`.
3. Implement the RMI server: `AssistantServer.java`.
 - The interface is “implemented”.
 - A client stub is generated and registered to the RMI registry.
 - The server listens on a server socket and accept incoming remote calls after stub generation.

```
J AssistantServer.java x
rpc_rest > 0.rpc > 1.java_rmi > server > J AssistantServer.java > ...
1  package server;
2
3  import java.rmi.registry.LocateRegistry;
4  import java.rmi.registry.Registry;
5  import java.rmi.server.UnicastRemoteObject;
6
7  import protocol.AssistantService;
8
9  public class AssistantServer implements AssistantService {
10     public AssistantServer() {
11     }
12
13     /* Implement remote procedures as a service. */
14
15     public String greetWithInfo(String userName, String institution) {
16         return String.format("Hello %s from %s!", userName, institution);
17     }
18
19     public double multiply(double xin, double yin) {
20         return xin * yin;
21     }
22
23     /*
24     * Main program to start a server and register the service to the RMI registry.
25     */
26     Run|Debug
27     public static void main(String[] args) {
28         try {
29             AssistantServer s = new AssistantServer();
30             // Create a stub for the assistant service.
31             // The corresponding remote object will accept incoming RMI calls on port=0,
32             // meaning the RMI system will automatically select an available TCP port.
33             AssistantService stub = (AssistantService) UnicastRemoteObject.exportObject(s, 0);
34             // Bind the remote object's stub in the registry
35             Registry registry = LocateRegistry.getRegistry("localhost", 1099);
36             registry.rebind("Assistant", stub);
37             System.out.println("Server ready.");
38         } catch (Exception e) {
39             System.err.println("Server exception: " + e.toString());
40             e.printStackTrace();
41         }
42     }
43
44     /**
45     * The server will not exit immediately because:
46     *
47     * "The static method UnicastRemoteObject.exportObject exports the supplied
48     * remote object to receive incoming remote method invocations on an anonymous
49     * TCP port and returns the stub for the remote object to pass to clients. As a
50     * result of the exportObject call, the runtime may begin to listen on a new
51     * server socket or may use a shared server socket to accept incoming remote
52     * calls for the remote object. The returned stub implements the same set of
53     * remote interfaces as the remote object's class and contains the host name and
54     * port over which the remote object can be contacted."
55     */
56 }
```

TASK: Java RMI - Hello World

Implement a Hello World Java RMI example.

> Reference codebase: [rpc_javarmi_hello_world](#)

1. Set up Java Development Kit (JDK) via:
 - o apt install openjdk-17-jdk
 - o Java RMI is a built-in library, and RMI Registry is natively implemented in the rmiregistry program.
2. Check the remote interface definition file AssistantService.java.
3. Implement the RMI server: AssistantServer.java.
4. Implement the RMI client: **Client.java**.
 - o The client stub is fetched from the RMI registry.
 - o RMI is performed using the client stub as if the remote object were local.

```

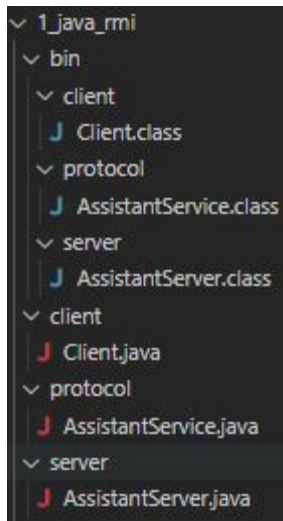
Client.java X
rpc:rest > 0.rpc > 1.java_rmi > client > J Client.java > ...
1  package client;
2
3  import java.rmi.registry.LocateRegistry;
4  import java.rmi.registry.Registry;
5
6  import protocol.AssistantService;
7
8  public class Client {
9      private Client() {}
10
11
12      Run | Debug
13      public static void main(String[] args) {
14          try {
15              // Find the client stub we want from the RMI registry
16              Registry registry = LocateRegistry.getRegistry("localhost", 1099);
17              AssistantService stub = (AssistantService) registry.lookup("Assistant");
18              /* RMI now! */
19              // Greeting
20              System.out.println("> Greet: Hello Assistant? I am \"Peter\" from \"SUSTech\".");
21              String greetMsg = stub.greetWithInfo(userName:"Peter", institution:"SUSTech");
22              System.out.println("> Client received: " + greetMsg);
23              // Multiplication
24              System.out.println("> Mult: Requesting a multiplication task - 3.5 x 5");
25              double multRes = stub.multiply(xdn:3.5, yin:5);
26              System.out.println("> Client received: " + multRes);
27          } catch (Exception e) {
28              System.err.println("Client exception: " + e.toString());
29              e.printStackTrace();
30          }
31      }
  
```

TASK: Java RMI - Hello World

Implement a Hello World Java RMI example.

> Reference codebase: [rpc_javarmi_hello_world](#)

1. Set up Java Development Kit (JDK) via:
 - `apt install openjdk-17-jdk`
 - Java RMI is a built-in library, and RMI Registry is natively implemented in the `rmiregistry` program.
2. Check the remote interface definition file `AssistantService.java`.
3. Implement the RMI server: `AssistantServer.java`.
4. Implement the RMI client: `Client.java`.
5. Compile the source files via:
 - `javac -d bin/ protocol/AssistantService.java server/AssistantServer.java client/Client.java`
 - The binaries will be output to the `bin/` folder.

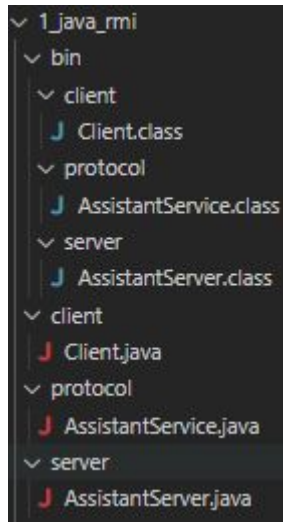


TASK: Java RMI - Hello World

Implement a Hello World Java RMI example.

> Reference codebase: `rpc_javarmi_hello_world`

- Run the RMI registry at port=1099 from the class path directory bin/:
 - `cd bin/`
 - `rmiregistry 1099`
 - The RMI registry **MUST** be started from the generated class path folder, otherwise the server and the client will fail to locate the defined interface so that they can marshal and unmarshal remote objects as required in Java RMI.
- Start the server in another terminal:
 - `java -cp bin/ server.AssistantServer`
- Execute the client in another terminal:
 - `java -cp bin/ client.Client`



```
java -cp bin/ server.AssistantServer
Server ready.
```

```
(base) root@RAINBOW: /rainbow/data/lab/0000/0000-1ab/cp
java -cp bin/ client.Client
> Greet: Hello Assistant? I am "Peter" from "SUSTech".
> Client received: Hello Peter from SUSTech!
> Mult: Requesting a multiplication task - 3.5 x 5
> Client received: 17.5
```

Java RMI - Design Concerns

- **Protocol:**
 - uses Java interfaces to specify the remote method signatures in an OOP fashion.
- **Communication:**
 - **Serialization:** object serialization via `java.io.Serializable`.
 - **Network Protocol:** naive Java RMI establishes a new TCP connection for each RMI request.
- **Proxy:**
 - **Dynamic Proxy** - client stubs & server stubs (skeletons) are dynamically generated at runtime utilizing Java Reflection.
 - Since client stubs are dynamically generated by the server, an additional **RMI registry** is needed to forward the stubs to the client.

```
J AssistantService.java X
rpc_rest > 0_rpc > 1_java_rmi > protocol > J AssistantService.java > ...
1 package protocol;
2
3 import java.rmi.Remote;
4 import java.rmi.RemoteException;
5
6 public interface AssistantService extends Remote {
7     String greetWithInfo(String userName, String institution) throws RemoteException;
8
9     double multiply(double xin, double yin) throws RemoteException;
10 }
11
12 /**
13  * To completely mimic the behavior of the proto file, where remote procedures
14  * may have multiple inputs and outputs, define the message classes manually in
15  * separate files from this package. Requests may be skipped since Java handles
16  * multiple inputs naturally.
17  */
```

```
J \javarm\src\java\rm\server\UnicastServerRef.class X
41 public class UnicastServerRef extends UnicastRef implements ServerRef, Dispatcher {
98     public Remote exportObject(Remote impl, Object data, boolean permanent) throws RemoteException {
99         Class<> implClass = impl.getClass();
100
101         Remote stub;
102         try {
103             stub = Util.createProxy(implClass, this.getClientRef(), this.forceStubUse);
104         } catch (IllegalArgumentException var7) {
105             throw new ExportException("remote object implements illegal remote interface", var7);
106         }
107
108         if (stub instanceof RemoteStub) {
109             this.setSkeleton(impl);
110         }
111
112         Target target = new Target(impl, this, stub, this.ref.getObjID(), permanent);
113         this.ref.exportObject(target);
114         this.hashToMethod_Map = (Map)hashToMethod_Maps.get(implClass);
115         return stub;
116     }
117
118     public String getClientHost() throws ServerNotActiveException { ...
119
120
121
122     public void setSkeleton(Remote impl) throws RemoteException {
123         if (!withoutSkeletons.containsKey(impl.getClass())) {
124             try {
125                 this.skel = Util.createSkeleton(impl);
126             } catch (SkeletonNotFoundException var3) {
127                 withoutSkeletons.put(impl.getClass(), (Object)null);
128             }
129         }
130     }
131 }
```

client stub (points to `Util.createProxy`)

server stub (skeleton) (points to `this.setSkeleton(impl)` and `Util.createSkeleton(impl)`)

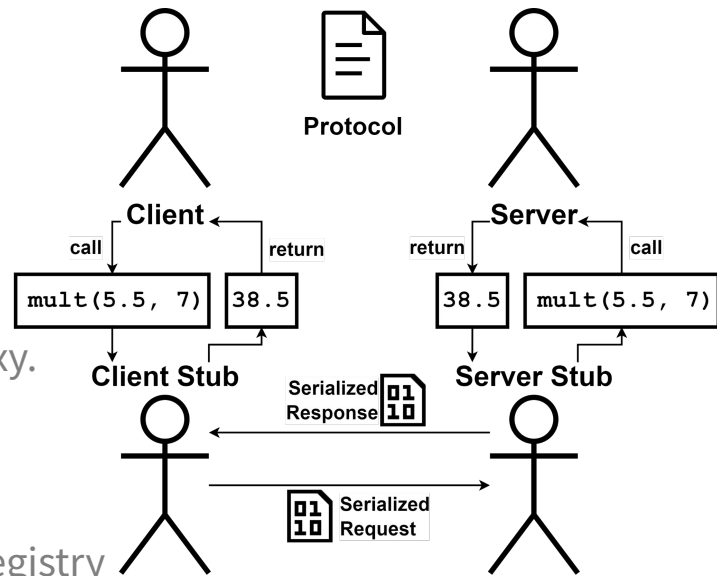
gRPC Compared to Java RMI

gRPC seems to be currently far more favorable than Java RMI, mainly because:

- **Cross-Language Support:**
 - Java RMI is Java-specific, but gRPC supports communication across different programming languages.
- **Serialization:**
 - Java Object Serialization is slower and more resource-intensive (due to its general-purpose design that records lots of object metadata) than Protobuf for gRPC.
- **Proxy Approach:**
 - Dynamic proxy from Java RMI reduces the codebase complexity, but at the same time introduces stub generation overhead at runtime, making the execution slightly slower than gRPC.

Summary

- Remote Procedure Call (RPC)
 - a. **Principles:**
 - i. serve remotely
 - ii. invoke as if it were local
 - iii. abstract remote interaction details
 - b. **Design Concerns:** Protocol, Communication, Proxy.
- Java RMI (Sun Microsystems, 1997)
 - a. Protocol: Java Interface
 - b. Communication: Object Serialization + TCP
 - c. Proxy: Dynamic Proxy via Java Reflection + RMI Registry
- **gRPC (Google, 2015)**
 - a. Protocol: Proto files from Protobuf
 - b. Communication: Protobuf + HTTP/2
 - c. Proxy: **protoc**
- gRPC Benefits: auto codegen; efficient Protobuf; ...



More details on gRPC in the next lab session...